

PHÂN LOẠI CIFAR-10 BẰNG DEEP LEARNING VÀ VISION TRANSFORMER

Hồ Võ Hoàng Duy - 25C01002

Ngày 3 tháng 10 năm 2026

Tóm tắt

Báo cáo so sánh AlexNet, VGG11-BN, ResNet18 và Vision Transformer (ViT tiny) trong bài toán phân loại ảnh CIFAR-10. Các mạng được điều chỉnh cho ảnh 32×32 và huấn luyện từ đầu trong 200 epoch, với 45.000 ảnh huấn luyện, 5.000 ảnh kiểm định và 10.000 ảnh kiểm tra. Kết quả được đối chiếu với các phương pháp dùng pixel thô, HOG và SIFT kết hợp SVM hoặc Random Forest. AlexNet đạt độ chính xác 83,87% và macro-F1 0,8381, cao nhất trong thí nghiệm; ViT tiny đạt 68,21%. Báo cáo tập trung vào khác biệt giữa CNN và ViT, chi phí huấn luyện và các lỗi phân loại thường gặp.

1 Giới thiệu

Phân loại ảnh là gán ảnh vào một trong các lớp đã định trước, chẳng hạn máy bay, ô tô hoặc mèo. CIFAR-10 gồm 60.000 ảnh màu 32×32 thuộc 10 lớp [7]. Độ phân giải nhỏ khiến nhiều chi tiết khó nhận biết, đặc biệt ở các lớp có hình dáng gần nhau.

Báo cáo khảo sát hai nhóm mô hình học sâu: mạng tích chập (CNN) và Vision Transformer. AlexNet kết hợp tích chập, ReLU và pooling; VGG xếp chồng các lớp tích chập nhỏ; ResNet bổ sung kết nối tắt để học phân dư [8, 10, 5]. ViT chia ảnh thành các vùng nhỏ (patch) và dùng self-attention để kết hợp thông tin giữa các vùng [4].

Mục tiêu là cài đặt bốn mạng, đánh giá độ chính xác (accuracy) và macro-F1, rồi so sánh với các phương pháp truyền thống. Qua đó, báo cáo xem xét lợi ích của việc học đặc trưng trực tiếp từ ảnh và những hạn chế của từng cấu hình.

2 Dữ liệu và tiền xử lý

CIFAR-10 có 50.000 ảnh huấn luyện và 10.000 ảnh kiểm tra chính thức. Tập huấn luyện được chia phân tầng với seed 42 thành 45.000 ảnh huấn luyện và 5.000 ảnh kiểm định, giữ số mẫu cân bằng giữa các lớp. Tập kiểm tra được giữ nguyên và chỉ dùng để đánh giá sau khi chọn bộ trọng số tốt nhất theo độ chính xác trên tập kiểm định.

Bảng 1: Phân chia dữ liệu cho các thí nghiệm.

Tập	Số ảnh	Số ảnh/lớp
Huấn luyện	45.000	4.500
Kiểm định	5.000	500
Kiểm tra	10.000	1.000

Đầu vào của mô hình học sâu. Ảnh giữ nguyên ba kênh RGB và kích thước 32×32 . Giá trị điểm ảnh được chia cho 255 để đưa về khoảng $[0, 1]$. Sau đó, ở mỗi kênh màu, giá trị này được trừ đi trung bình và chia cho độ lệch chuẩn:

$$\hat{I}_{c,h,w} = \frac{I_{c,h,w}/255 - \mu_c^{\text{train}}}{\max(\sigma_c^{\text{train}}, 10^{-6})}. \quad (1)$$

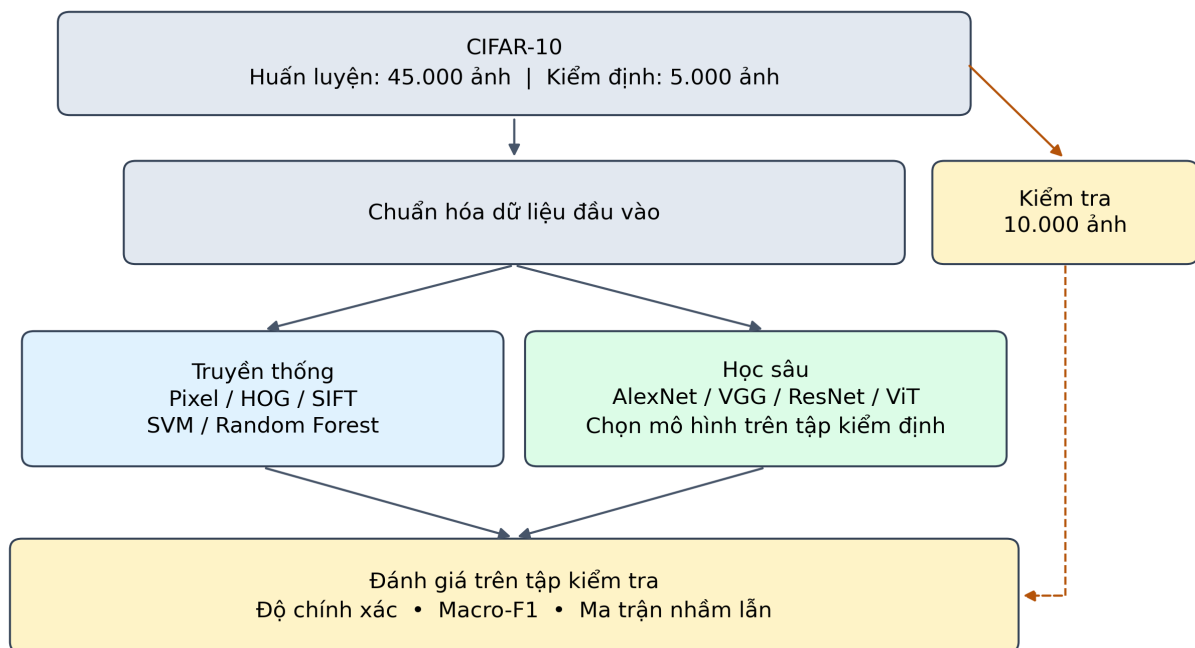
Trong công thức, μ_c^{train} và σ_c^{train} là trung bình và độ lệch chuẩn của kênh màu c , tính từ tập huấn luyện. Các giá trị này được giữ nguyên khi xử lý ảnh kiểm định và kiểm tra. Thí nghiệm không dùng tăng cường dữ liệu.

Đầu vào của mô hình truyền thống. Pixel thô được trải phẳng thành vector 3.072 chiều. HOG và SIFT dùng ảnh xám $I = 0.299R + 0.587G + 0.114B$. Mỗi chiều của vector đặc trưng cũng được trừ đi trung bình và chia cho độ lệch chuẩn tính từ tập huấn luyện.



Hình 1: Ảnh mẫu của 10 lớp CIFAR-10, lấy từ tập huấn luyện.

2.1 Quy trình thực nghiệm



Hình 2: Quy trình chuẩn hóa dữ liệu, huấn luyện và đánh giá hai nhóm mô hình.

Bước chuẩn hóa điều chỉnh thang giá trị của dữ liệu đầu vào theo cách đã mô tả ở trên. Nhánh truyền thống trích xuất đặc trưng rồi huấn luyện SVM hoặc Random Forest. Nhánh học sâu học đồng thời đặc trưng và bộ phân lớp. Cả hai được đánh giá trên cùng tập kiểm tra để so sánh kết quả.

3 Phương pháp học sâu

Các mạng được điều chỉnh cho ảnh CIFAR-10 và học trọng số từ đầu. Kích thước tensor được ghi là $C \times H \times W$ cho CNN (kênh, chiều cao, chiều rộng) và $T \times D$ cho ViT (số token, số chiều biểu diễn), bỏ qua chiều batch.

3.1 Nguyên lý chung

Tích chập. Lớp tích chập kết hợp các điểm ảnh lân cận trên các kênh đầu vào để tạo đặc trưng:

$$a_{o,h,w} = b_o + \sum_{c=1}^{C_{in}} \sum_{u,v} W_{o,c,u,v} x_{c,sh+u-p,sw+v-p}. \quad (2)$$

Trong đó, W là bộ lọc (kernel), s là bước trượt (stride) và p là phần đệm (padding). Bộ lọc được dùng chung ở nhiều vị trí, giúp nhận ra cùng một mẫu cạnh hoặc kết cấu ở các vùng khác nhau. Kernel 3×3 , padding 1 và stride 1 giữ nguyên kích thước ảnh; stride 2 giảm khoảng một nửa.

ReLU và pooling. ReLU, $\max(0, a)$, giúp mạng học các quan hệ phi tuyến. Max-pooling 2×2 chọn giá trị lớn nhất trong mỗi vùng để giảm kích thước. Global average pooling lấy trung bình trên toàn bộ vùng không gian của từng kênh. Qua nhiều lớp, mạng kết hợp các mẫu cục bộ thành đặc trưng của những vùng ảnh lớn hơn.

Học đặc trưng. Mạng được viết thành $z = g_\theta(f_\phi(I))$, với f_ϕ là bộ trích xuất đặc trưng, g_θ là bộ phân lớp và z gồm 10 điểm số đầu ra (logit). Cả hai bộ phận được cập nhật theo nhãn huấn luyện, thay vì giữ cố định đặc trưng như HOG hay SIFT.

Huấn luyện và đánh giá. Softmax chuyển logit thành điểm xác suất; hàm mất mát (loss) cross-entropy đo sai lệch so với nhãn thật:

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{10} \exp(z_{ij})}, \quad \mathcal{L}_{CE} = -\frac{1}{B} \sum_{i=1}^B \log p_{i,y_i}. \quad (3)$$

Ở đây, B là số ảnh trong batch và y_i là nhãn thật. Mỗi batch được đưa qua mạng, tính loss, lan truyền ngược và cập nhật trọng số bằng AdamW. Sau mỗi epoch, mô hình được đánh giá trên tập kiểm định và lưu bộ trọng số có accuracy cao nhất. Khi suy luận, dropout được tắt, BatchNorm dùng thống kê tích lũy và nhãn dự đoán là $\hat{y} = \arg \max_k z_k$.

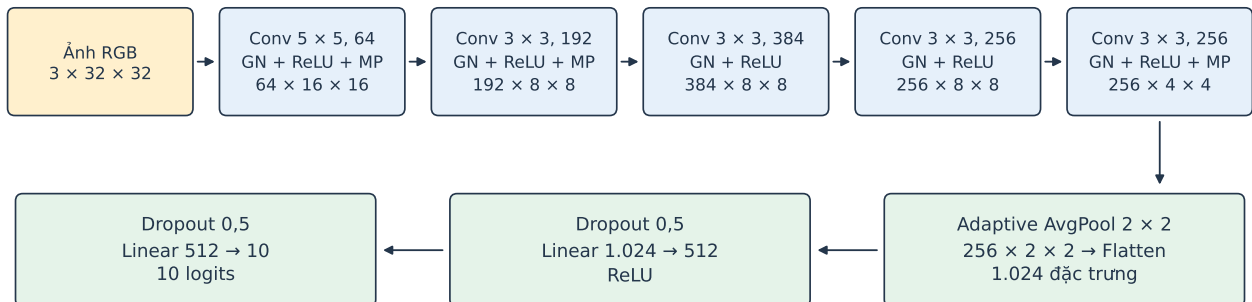
3.2 AlexNet với GroupNorm

Kiến trúc. AlexNet kết hợp tích chập, ReLU, pooling và bộ phân lớp có dropout [8]. Biến thể CIFAR-10 gồm năm lớp tích chập, mỗi lớp theo sau bởi GroupNorm và ReLU. Các lớp dùng stride 1; padding 2 cho kernel 5×5 và padding 1 cho kernel 3×3 . Vì vậy, chỉ các lớp pooling làm giảm kích thước không gian.

Bảng 2: Các bước xử lý của AlexNet. Conv là lớp tích chập, theo sau bởi GroupNorm và ReLU.

Bước	Thao tác	Đầu ra
Đầu vào	RGB đã chuẩn hóa	$3 \times 32 \times 32$
1	Conv 5×5 , 64 kênh; max-pool	$64 \times 16 \times 16$
2	Conv 3×3 , 192 kênh; max-pool	$192 \times 8 \times 8$
3	Conv 3×3 , 384 kênh	$384 \times 8 \times 8$
4	Conv 3×3 , 256 kênh	$256 \times 8 \times 8$
5	Conv 3×3 , 256 kênh; max-pool	$256 \times 4 \times 4$
6	Adaptive average pooling 2×2 ; flatten	1024
7	Dropout; Linear $1024 \rightarrow 512$; ReLU	512
8	Dropout; Linear $512 \rightarrow 10$	10 logit

AlexNet CIFAR-10 với GroupNorm



GN: GroupNorm, 8 nhóm • MP: MaxPool 2×2 , stride 2 • Conv: stride 1, padding giữ kích thước

Hình 3: Kiến trúc AlexNet cho CIFAR-10 với GroupNorm và dropout.

Chuẩn hóa và điều chuẩn. GroupNorm chia các kênh thành tám nhóm và chuẩn hóa từng nhóm trên mỗi ảnh, nên không phụ thuộc số ảnh trong batch [12]. Adaptive pooling tạo lưới 2×2 , sau đó hai lớp tuyến tính thực hiện phân loại. Dropout 0,5 ở bộ phân lớp giúp hạn chế sự phụ thuộc vào một số đặc trưng.

Mạng có 2.786.890 tham số. Kích thước bộ lọc, các bước giảm kích thước và bộ phân lớp đã được điều chỉnh cho ảnh nhỏ, nên cấu hình này khác AlexNet nguyên bản trên ImageNet.

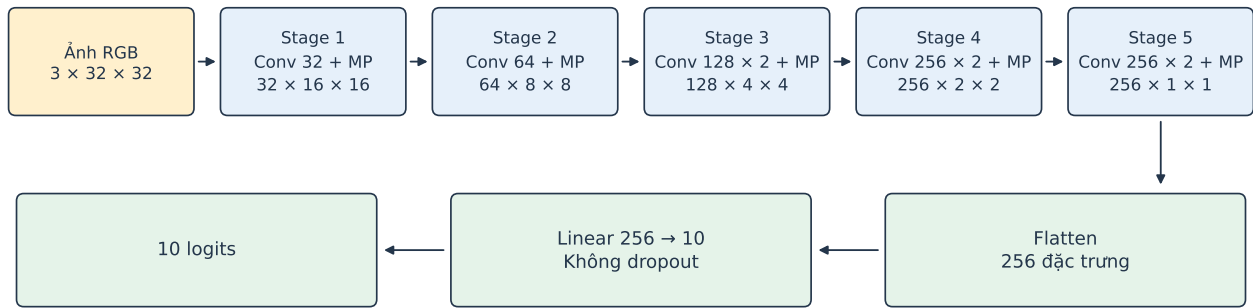
3.3 VGG11-BN với số kênh giảm

Kiến trúc. VGG dùng các lớp tích chập 3×3 xếp chồng [10]. Bản cài đặt gồm tám lớp, mỗi lớp dùng padding 1, stride 1, BatchNorm và ReLU. Năm bước max-pooling giảm dần kích thước không gian từ 32×32 về 1×1 .

Bảng 3: Năm khối xử lý của VGG11-BN, mỗi khối kết thúc bằng max-pooling 2×2 .

Khối	Các lớp tích chập 3×3	Đầu ra sau pooling	Số lớp
1	$3 \rightarrow 32$	$32 \times 16 \times 16$	1
2	$32 \rightarrow 64$	$64 \times 8 \times 8$	1
3	$64 \rightarrow 128 \rightarrow 128$	$128 \times 4 \times 4$	2
4	$128 \rightarrow 256 \rightarrow 256$	$256 \times 2 \times 2$	2
5	$256 \rightarrow 256 \rightarrow 256$	$256 \times 1 \times 1$	2

VGG11-BN CIFAR-10 với số kênh giảm



Mỗi Conv: kernel 3×3 , stride 1, padding 1 → BatchNorm → ReLU • MP: MaxPool 2×2 , stride 2

Hình 4: Kiến trúc VGG11-BN với tám lớp tích chập và năm bước max-pooling.

Chuẩn hóa và phân lớp. BatchNorm chuẩn hóa đầu ra từng kênh bằng thống kê của batch khi huấn luyện và dùng thống kê tích lũy khi suy luận [6]. Sau khối cuối, 256 đặc trưng được đưa vào một lớp tuyến tính để dự đoán 10 lớp; mạng không dùng dropout.

VGG11-BN có 2.310.186 tham số, ít nhất trong bốn mạng. Cấu trúc tuần tự dễ triển khai, nhưng nhiều bước giảm kích thước có thể làm mất chi tiết trên ảnh nhỏ.

3.4 ResNet18 với kết nối phần dư

BasicBlock. Mỗi block chứa hai lớp tích chập 3×3 kèm BatchNorm; ReLU đặt sau lớp tích chập thứ nhất và sau phép cộng với nhánh tắt:

$$F(x) = \text{BN}_2(\text{Conv}_2(\text{ReLU}(\text{BN}_1(\text{Conv}_1(x))))), \quad (4)$$

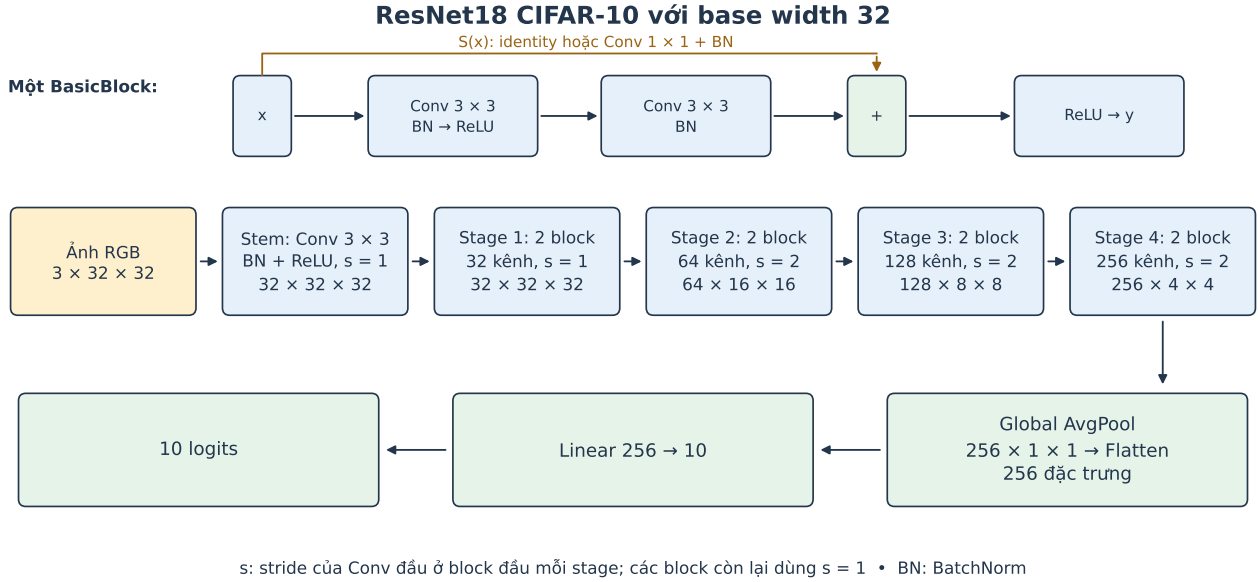
$$y = \text{ReLU}(F(x) + S(x)). \quad (5)$$

Khi số kênh và kích thước không đổi, $S(x) = x$. Khi thay đổi kích thước hoặc số kênh, nhánh tắt dùng tích chập 1×1 và BatchNorm để hai nhánh có cùng kích thước trước phép cộng.

Vai trò của kết nối phần dư. Khi nhánh tắt giữ nguyên đầu vào, mạng học phần thay đổi $F(x)$ rồi cộng lại với x , thay vì học toàn bộ ảnh xạ mới. Đường truyền trực tiếp này giúp gradient đi qua nhiều lớp thuận lợi hơn [5].

Bảng 4: Các khối chính của ResNet18; mỗi khối có hai BasicBlock.

Phần	Số kênh	Stride block đầu	Đầu ra
Stem: Conv 3×3 + BN + ReLU	32	1	$32 \times 32 \times 32$
Khối 1	32	1	$32 \times 32 \times 32$
Khối 2	64	2	$64 \times 16 \times 16$
Khối 3	128	2	$128 \times 8 \times 8$
Khối 4	256	2	$256 \times 4 \times 4$
Global average pooling; linear	256	–	$256 \rightarrow 10$



Hình 5: Kiến trúc ResNet18 và hai nhánh của một BasicBlock.

Điều chỉnh cho ảnh nhỏ. Lớp tích chập đầu dùng kernel 3×3 , stride 1 và không có max-pooling, giúp giữ độ phân giải ở đầu mạng. Bốn khối chính gồm tám BasicBlock; số kênh bắt đầu từ 32. Global average pooling tạo 256 đặc trưng trước lớp phân loại. Mạng có 2.797.610 tham số.

3.5 Vision Transformer: ViT tiny

Tạo token từ ảnh. Ảnh được chia thành 64 patch 4×4 không chồng lấn. Mỗi patch được ánh xạ thành vector 192 chiều (token). Mạng thêm một class token để tổng hợp thông tin phân loại và embedding vị trí để ghi nhận vị trí của các patch [4]:

$$Z_0 = [x_{cls}; x_p^1 E; \dots; x_p^{64} E] + E_{pos}, \quad E \in \mathbb{R}^{48 \times 192}, \quad E_{pos} \in \mathbb{R}^{65 \times 192}. \quad (6)$$

Trong công thức, E là ma trận chiều patch (lược bỏ bias), E_{pos} là embedding vị trí; chuỗi đầu vào có kích thước 65×192 .

Self-attention. Mỗi block dùng LayerNorm và ba attention head. Các phép chiếu tuyến tính tạo truy vấn Q , khóa K và giá trị V . Mỗi head có $Q_h, K_h, V_h \in \mathbb{R}^{65 \times 64}$:

$$A_h = \text{softmax}_{\text{hàng}} \left(\frac{Q_h K_h^T}{\sqrt{64}} \right), \quad U_h = A_h V_h. \quad (7)$$

A_h chứa trọng số kết hợp thông tin giữa các token. Đầu ra của ba head được nối lại và chiếu về 192 chiều. Nhờ đó, mỗi token có thể nhận thông tin từ toàn bộ ảnh ngay trong một block.

Các block encoder. Mạng dùng sáu block, mỗi block gồm attention, MLP và hai kết nối phần dư. LayerNorm được đặt trước attention và MLP:

$$Z'_\ell = Z_{\ell-1} + \text{MSA}(\text{LN}(Z_{\ell-1})), \quad (8)$$

$$Z_\ell = Z'_\ell + \text{MLP}(\text{LN}(Z'_\ell)), \quad (9)$$

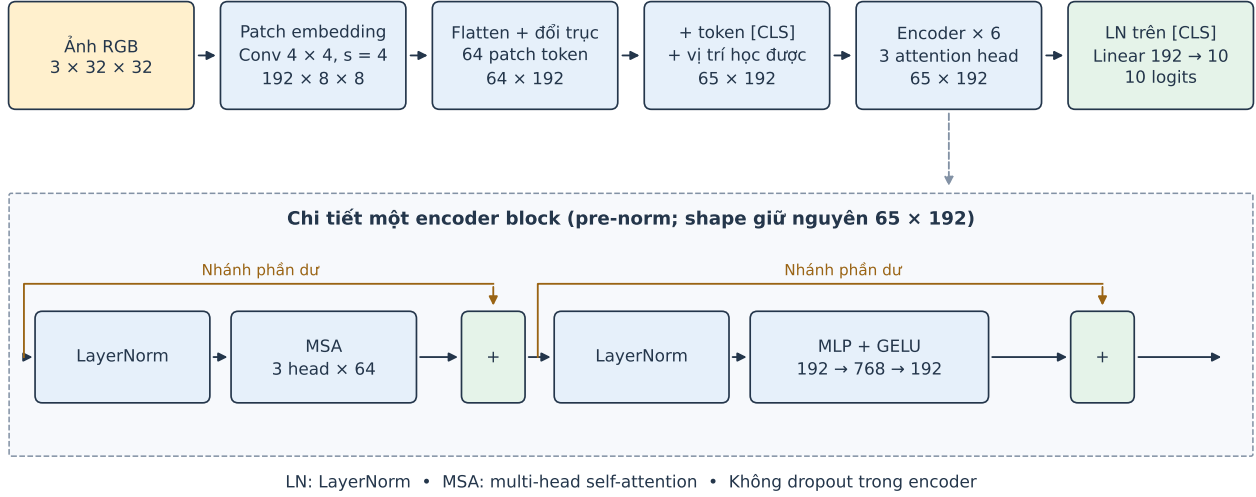
$$\text{MLP}(u) = W_2 \text{GELU}(W_1 u + b_1) + b_2. \quad (10)$$

MLP mở rộng $192 \rightarrow 768$ rồi thu về 192 cho từng token. Kích thước chuỗi 65×192 được giữ nguyên qua sáu block.

Bảng 5: Các bước xử lý và kích thước đầu ra của ViT tiny.

Bước	Cấu hình	Đầu ra
Patch embedding	Conv kernel 4, stride 4, 192 kênh	$192 \times 8 \times 8$
Chuỗi token	Flatten lưới; thêm class token và vị trí	65×192
Encoder	6 block; 3 head; MLP ratio 4	65×192
Đầu phân lớp	LayerNorm; lấy class token; Linear	$192 \rightarrow 10$

ViT tiny CIFAR-10: patch 4, embedding 192, 6 block



Hình 6: Kiến trúc ViT tiny và cấu trúc một block encoder.

Phân lớp. Sau encoder, class token được chuẩn hóa bằng LayerNorm và đưa vào lớp tuyến tính tạo 10 logit. Mạng có 2.693.578 tham số và không dùng dropout trong các block. Chi phí attention tăng theo bình phương số token, nên dùng patch nhỏ hơn sẽ tăng chi phí tính toán.

3.6 Phương pháp truyền thống

Pixel thô được trải phẳng thành vector 3.072 chiều. HOG chia ảnh xám thành cell 4×4 , tính histogram 9 hướng gradient, chuẩn hóa trên block 2×2 chồng lấn và nối thành 1.764 chiều [3]. HOG mô tả bố cục hướng cạnh trên lưới ảnh.

SIFT phẳng dùng bản cài đặt rút gọn với một octave, tìm các điểm đặc trưng (keypoint) và mô tả mỗi điểm bằng vector 128 chiều. Tối đa 64 vector mô tả (descriptor) được xếp theo độ mạnh, nối và đếm số 0 thành 8.192 chiều [9]. Cách này không dùng Bag of Visual Words; thứ tự keypoint khác nhau giữa các ảnh có thể khiến các chiều đặc trưng thiếu nhất quán.

SVM tuyến tính học các điểm số $s_k = w_k^T x + b_k$ bằng hinge loss đa lớp và điều chuẩn L2 [2]. Random Forest huấn luyện nhiều cây trên các mẫu lấy ngẫu nhiên có hoàn lại, chọn ngẫu nhiên đặc trưng khi chia nút và bỏ phiếu nhân [1]. Hai bộ phân lớp được kết hợp với ba loại đặc trưng để tạo sáu cấu hình so sánh.

4 Thiết lập thí nghiệm

Bốn mạng được cài đặt bằng các lớp cơ bản của PyTorch, khởi tạo ngẫu nhiên và huấn luyện độc lập, không dùng trọng số huấn luyện sẵn. Mỗi mạng chạy 200 epoch với seed 42, AdamW, tốc độ học 3×10^{-4} , weight decay 10^{-4} và batch size 128. Tốc độ học được giữ cố định; chuẩn gradient được giới hạn ở 1,0.

SVM chạy 200 epoch với batch size 256, tốc độ học ban đầu 0,01 và hệ số điều chuẩn 10^{-4} . Random Forest dùng 200 cây, độ sâu tối đa 15, tối thiểu 4 mẫu để chia nút và 2 mẫu ở nút lá. Mỗi nút xét 16 ngưỡng trên $\sqrt{d}$ chiều đặc trưng chọn ngẫu nhiên.

Với ma trận nhầm lẫn C_{ij} (hàng là nhãn thật, cột là nhãn dự đoán), accuracy, precision, recall và F1 được tính như sau:

$$\text{Accuracy} = \frac{\sum_k C_{kk}}{\sum_{i,j} C_{ij}}, \quad P_k = \frac{C_{kk}}{\sum_i C_{ik}}, \quad R_k = \frac{C_{kk}}{\sum_j C_{kj}}, \quad F1_k = \frac{2P_k R_k}{P_k + R_k}. \quad (11)$$

Accuracy đo tỷ lệ ảnh được phân loại đúng. Macro-F1 là trung bình F1 của 10 lớp, cho mỗi lớp trọng số ngang nhau. Nếu mẫu số bằng 0, chỉ số tương ứng được đặt bằng 0. Các chỉ số trong bảng được trình bày trên thang $[0, 1]$.

5 Kết quả và thảo luận

5.1 Bảng so sánh tổng hợp

Bảng 6: Kết quả trên 10.000 ảnh kiểm tra CIFAR-10.

Nhóm	Cấu hình	N_{test}	Accuracy	Macro-F1
Truyền thống	Pixel-SVM	10000	0.3757	0.3751
Truyền thống	Pixel-RF	10000	0.4509	0.4462
Truyền thống	HOG-SVM	10000	0.5319	0.5305
Truyền thống	HOG-RF	10000	0.4673	0.4593
Truyền thống	SIFT-SVM	10000	0.2292	0.2299
Truyền thống	SIFT-RF	10000	0.1857	0.1433
Học sâu	AlexNet (GroupNorm)	10000	0.8387	0.8381
Học sâu	VGG11-BN	10000	0.7952	0.7944
Học sâu	ResNet18	10000	0.8145	0.8141
Học sâu	ViT tiny	10000	0.6821	0.6803

Bảng 6 cho thấy cả bốn mạng học sâu đều đạt kết quả cao hơn các phương pháp truyền thống. AlexNet đứng đầu với accuracy 83,87% và macro-F1 0,8381, tiếp theo là ResNet18 (81,45%), VGG11-BN (79,52%) và ViT tiny (68,21%). Trong nhóm truyền thống, HOG-SVM đạt kết quả tốt nhất với accuracy 53,19% và macro-F1 0,5305.

5.2 Kết quả huấn luyện các mô hình học sâu

Bảng 7: Epoch được chọn theo accuracy kiểm định và tổng thời gian huấn luyện 200 epoch.

Mô hình	Tham số	Epoch chọn	Acc. kiểm định	Thời gian (giờ)
AlexNet (GroupNorm)	2,786,890	193	0.8518	3.48
VGG11-BN	2,310,186	130	0.8120	2.19
ResNet18	2,797,610	184	0.8262	6.51
ViT tiny	2,693,578	180	0.6888	5.18

VGG11-BN huấn luyện nhanh nhất (2,19 giờ), còn ResNet18 mất nhiều thời gian nhất (6,51 giờ). AlexNet đạt accuracy cao nhất với thời gian 3,48 giờ.

5.3 Ý nghĩa của kết quả so sánh

Trên 10.000 ảnh kiểm tra, AlexNet phân loại sai 1.613 ảnh, trong khi HOG-SVM sai 4.681 ảnh. Như vậy, AlexNet có ít hơn 3.068 lỗi. Accuracy và macro-F1 của bốn mạng học sâu khá gần nhau, cho thấy ưu thế vẫn thể hiện khi đánh giá các lớp với trọng số ngang nhau. Tuy nhiên, chất lượng giữa từng lớp vẫn khác nhau, như phân tích lỗi của AlexNet ở phần sau.

Ưu thế của học sâu có thể được giải thích qua cách xây dựng đặc trưng. HOG và SIFT dùng quy tắc trích xuất cố định, còn CNN và ViT điều chỉnh đặc trưng theo mục tiêu phân loại. Các mạng nhận ảnh RGB, kết hợp màu sắc với cấu trúc không gian và học những quan hệ phi tuyến qua nhiều tầng. Trong khi đó, HOG và SIFT ở đây dùng ảnh xám; SVM chỉ học ranh giới tuyến tính trên các vector đầu vào.

SIFT-SVM và SIFT-Random Forest đạt kết quả thấp nhất. Cách nối trực tiếp các descriptor khiến cùng một vị trí trong vector có thể mô tả các vùng khác nhau giữa hai ảnh. Đây là hạn chế của cách biểu diễn SIFT phẳng đang dùng; các cách tổng hợp như Bag of Visual Words cần được đánh giá riêng.

Các mô hình còn khác nhau về số tham số, cách tối ưu và điều chuẩn. Vì vậy, bảng kết quả phản ánh hiệu quả của từng cấu hình hoàn chỉnh, chưa tách được đóng góp của riêng từng thành phần.

5.4 Vì sao ViT tiny đạt kết quả thấp hơn CNN?

ViT tiny đạt accuracy 68,21%, trong khi ResNet18 đạt 81,45% và VGG11-BN đạt 79,52%. Khác biệt này xuất hiện ở cả 10 lớp. Đáng chú ý, ViT nhầm mèo và chó tổng cộng 366 ảnh, so với 260 ảnh của ResNet18 và 250 ảnh của VGG11-BN. Ba yếu tố sau giúp giải thích kết quả và định hướng cải thiện.

Cách khai thác cấu trúc ảnh. CNN dùng các bộ lọc cục bộ và chia sẻ trọng số giữa nhiều vị trí. Cấu trúc này giúp mô hình học cạnh, kết cấu rồi kết hợp thành các đặc trưng lớn hơn. ViT chia ảnh thành patch và học quan hệ giữa chúng bằng attention, nên có ít giả định sẵn hơn về quan hệ lân cận trong ảnh [4]. Khi huấn luyện từ đầu với 45.000 ảnh, đặc điểm này

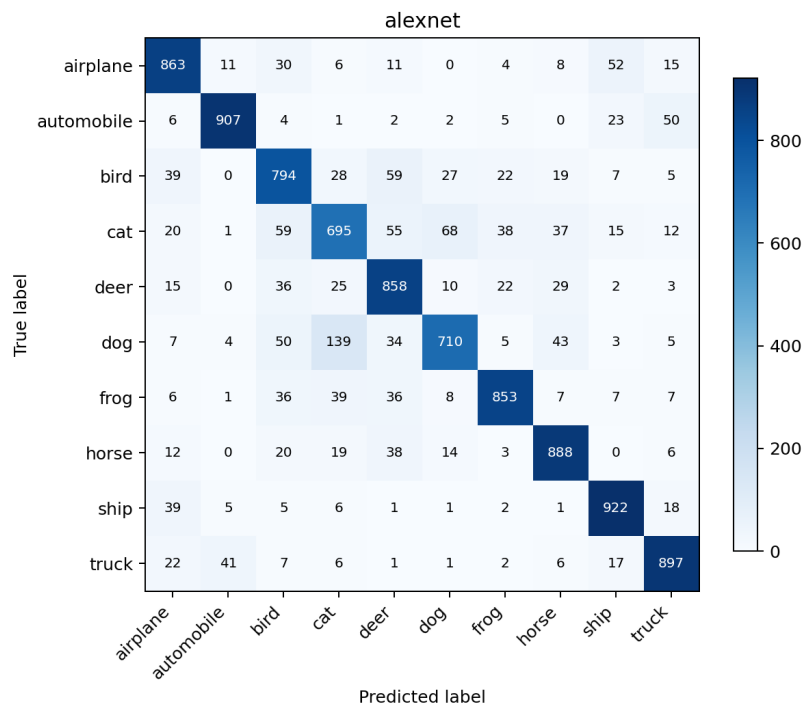
có thể khiến ViT khó học các quy luật thị giác hơn CNN. Patch 4×4 cũng không chồng lấn, nên đường viền đi qua ranh giới patch phải được kết hợp ở các block sau.

Khả năng tổng quát hóa còn hạn chế. Ở epoch tốt nhất (180), ViT có loss huấn luyện 0,0174 nhưng loss kiểm định 2,3826, với accuracy kiểm định 68,88%. Đến epoch 200, loss huấn luyện tiếp tục giảm còn 0,0141, trong khi loss kiểm định tăng lên 2,5317 và accuracy giảm còn 68,02%. Diễn biến này cho thấy dấu hiệu quá khớp: mô hình học tốt dữ liệu huấn luyện nhưng chưa nhận diện tốt ảnh mới. Chỉ tăng số epoch có thể không giải quyết được vấn đề.

Cách huấn luyện chưa được tối ưu riêng. Cả bốn mạng dùng cùng tốc độ học cố định và không có tăng cường dữ liệu. Thiết lập chung giúp so sánh trong cùng điều kiện, nhưng chưa chắc phù hợp nhất với từng kiến trúc. Nghiên cứu DeiT cho thấy tăng cường dữ liệu và điều chuẩn có vai trò quan trọng khi huấn luyện transformer [11]. ViT tiny trong thí nghiệm có khoảng 2,69 triệu tham số, gần ResNet18 và nhiều hơn VGG11-BN, nên số tham số ít không phải lời giải thích phù hợp cho kết quả thấp hơn.

Các yếu tố trên là những hướng giải thích, chưa được kiểm chứng bằng thí nghiệm thay đổi từng thành phần. Bước tiếp theo nên ưu tiên tăng cường dữ liệu, thử lịch giảm tốc độ học và điều chỉnh weight decay; sau đó khảo sát patch nhỏ hơn hoặc lớp tích chập trước khi tạo token. Cấu hình được chọn trên tập kiểm định và cần chạy lại với nhiều seed. Kết quả hiện tại chỉ cho thấy ViT tiny kém hơn CNN trong thiết lập này, chưa đủ để xếp hạng chung các kiến trúc.

5.5 Phân tích lỗi của mô hình tốt nhất



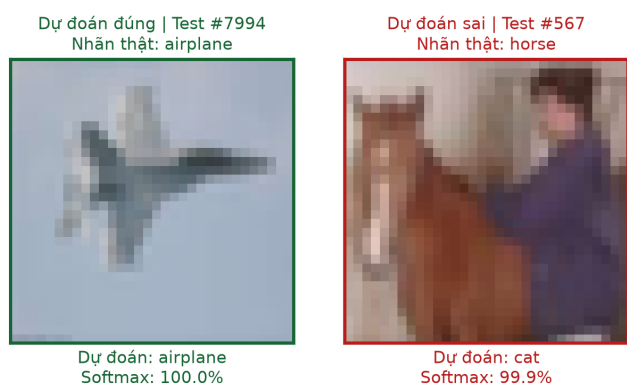
Hình 7: Ma trận nhầm lẫn của AlexNet: hàng là nhãn thật, cột là nhãn dự đoán; mỗi lớp có 1.000 ảnh kiểm tra.

AlexNet nhận diện tốt nhất lớp ô tô (*automobile*, F1 0,9208) và tàu thủy (*ship*, F1 0,9004). Lớp mèo (*cat*) khó nhất, với F1 0,7077 và recall 69,5%. Lỗi phổ biến nhất là nhầm chó thành mèo (139 ảnh); chiều ngược lại có 68 ảnh. Kết quả cho thấy mô hình vẫn khó phân biệt một số lớp động vật trên ảnh có độ phân giải thấp.

5.6 Minh họa dự đoán đúng và sai

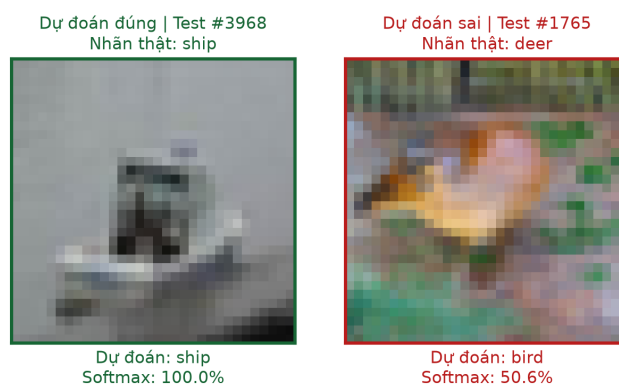
Mỗi mô hình được minh họa bằng một ảnh dự đoán đúng (trái, viền xanh) và một ảnh dự đoán sai (phải, viền đỏ) từ tập kiểm tra. Trong mỗi nhóm đúng hoặc sai, ảnh được sắp theo cross-entropy và chọn mẫu ở giữa để minh họa. Điểm softmax trên hình thuộc lớp dự đoán; điểm cao vẫn có thể đi kèm nhãn sai. Các ảnh này bổ sung góc nhìn trực quan cho kết quả tổng hợp trong Bảng 6.

AlexNet (GroupNorm)



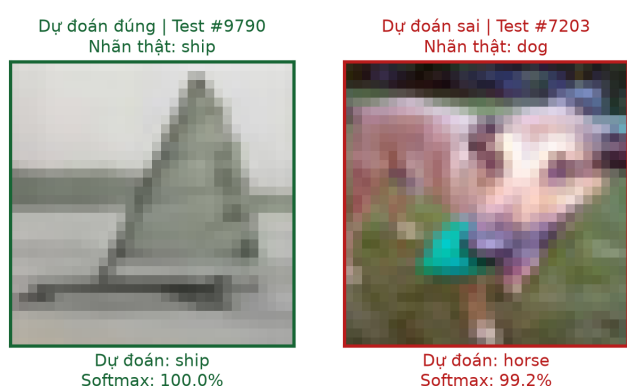
Hình 8: Dự đoán của AlexNet ở epoch 193.

VGG11-BN



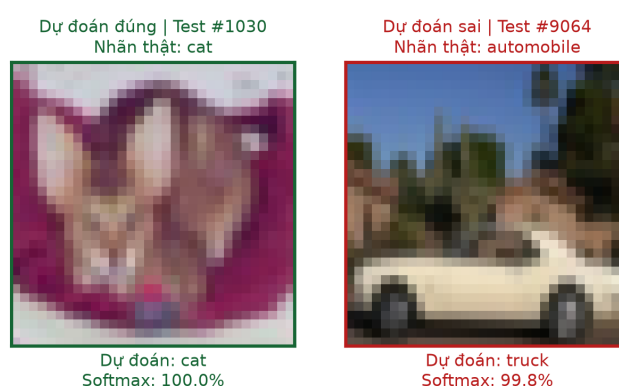
Hình 9: Dự đoán của VGG11-BN ở epoch 130.

ResNet18



Hình 10: Dự đoán của ResNet18 ở epoch 184.

ViT tiny



Hình 11: Dự đoán của ViT tiny ở epoch 180.

6 Kết luận

Bốn mô hình học sâu đều vượt các phương pháp truyền thống trong thí nghiệm này. AlexNet đạt kết quả tốt nhất với accuracy 83,87% và macro-F1 0,8381. Các CNN cũng đạt độ chính xác cao hơn ViT tiny khi huấn luyện từ đầu trên CIFAR-10 và không dùng tăng cường dữ liệu.

Kết quả cho thấy lợi ích của việc học đặc trưng trực tiếp từ ảnh, nhưng chỉ phản ánh một cách chia dữ liệu và một bộ siêu tham số. Hướng cải thiện chính là tăng cường dữ liệu, điều chỉnh tốc độ học và điều chuẩn trên tập kiểm định, sau đó lặp lại thí nghiệm với nhiều seed để đánh giá độ ổn định.

Tài liệu tham khảo

- [1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [2] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.
- [3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2005.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, 2015.
- [7] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [8] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [9] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [10] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.
- [11] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 10347–10357, 2021.
- [12] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision*, 2018.